Projet de fin d'études

Réalité augmentée & Intelligence artificielle

NeurAR

Visualisation en réalité augmentée d'un réseau de neurones en fonctionnement

Réalisé par : CHAIBOU SAIDOU ABDOULAYE

Encadré par : Pr. Tawfik Masrour
Pr. Ajebli Aymane
Pr. Maalouf Imad

Année universitaire 2025 – 2026

Résumé

Les réseaux de neurones sont aujourd'hui au cœur de la plupart des applications d'intelligence artificielle, mais leur fonctionnement interne reste difficile à appréhender pour les étudiants et les personnes découvrant le domaine. Le projet **NeurAR** propose une solution originale à ce problème : visualiser, en **réalité augmentée**, un réseau de neurones de type *Multi-Layer Perceptron* (MLP) en train de fonctionner, couche par couche, en temps réel.

Le système repose sur trois composants principaux : un **modèle PyTorch** entraîné sur le jeu de données MNIST, un **backend FastAPI** exposant une API REST et un canal WebSocket temps réel, et une **application Unity** en réalité augmentée qui reçoit ces données et les transforme en une représentation 3D animée et interactive. L'utilisateur peut ainsi observer en direct la propagation avant (*forward pass*) et la rétropropagation (*backward pass*) du réseau, manipuler la scène en AR (rotation, zoom, sélection de neurone) et suivre l'évolution de la perte et de la précision du modèle.

Ce rapport présente le contexte et les objectifs du projet, l'architecture technique mise en place, les choix d'implémentation pour le modèle de réseau de neurones, le protocole de communication temps réel, ainsi que l'application Unity AR. Il revient également sur les principales difficultés rencontrées, les solutions apportées, et propose des pistes d'amélioration pour la suite du projet.

Table des matières

Résumé **1**

1 Introduction **4**
1.1 Contexte 4
1.2 Problématique 4
1.3 Objectifs du projet 4
1.4 Plan du rapport 5

2 Architecture générale de la solution **6**
2.1 Vue d'ensemble 6
2.2 Rôle de chaque composant 6
2.2.1 Le modèle PyTorch 6
2.2.2 Le backend FastAPI 6
2.2.3 L'application Unity (client AR) 6
2.3 Technologies utilisées 7

3 Le modèle de réseau de neurones et le backend **8**
3.1 Architecture du réseau (NeuroARNet) 8
3.2 Suivi des activations, poids et biais 8
3.3 Entraînement et métriques 9
3.4 Modélisation des données échangées 9
3.5 API REST 9

4 Communication temps réel **10**
4.1 Le canal WebSocket 10
4.2 Scénario d'animation du forward pass 10
4.3 Gestion de la taille des messages 10

5 Application Unity en réalité augmentée **12**
5.1 Vue d'ensemble 12
5.2 Chaîne de traitement côté Unity 12
5.3 Représentation visuelle 13

6 Fonctionnalités réalisées et apports du projet **14**
6.1 Fonctionnalités réalisées 14
6.2 Apports et points forts du projet 14

7 Difficultés rencontrées et solutions apportées **15**

8 Conclusion et perspectives **16**
8.1 Conclusion 16
8.2 Perspectives d'amélioration 16

1. Introduction

1.1 Contexte

L'intelligence artificielle, et en particulier l'apprentissage profond (*deep learning*), occupe une place de plus en plus importante dans de nombreux domaines : reconnaissance d'images, traitement du langage, systèmes de recommandation, etc. Cependant, les réseaux de neurones sont souvent perçus comme des **boîtes noires** : on connaît leurs entrées et leurs sorties, mais leur fonctionnement interne (propagation des activations, mise à jour des poids) reste abstrait, surtout pour les personnes qui découvrent le domaine.

Par ailleurs, la **réalité augmentée (RA)** permet de superposer des objets virtuels au monde réel et offre de nouvelles possibilités pédagogiques : manipulation d'objets 3D, immersion, interaction naturelle. Combiner ces deux univers – intelligence artificielle et réalité augmentée – ouvre la voie à des outils pédagogiques innovants pour mieux comprendre le fonctionnement des modèles d'apprentissage automatique.

1.2 Problématique

Comment permettre à un utilisateur de **visualiser et de manipuler**, en temps réel et en trois dimensions, le fonctionnement interne d'un réseau de neurones (propagation des activations, poids, biais, métriques d'apprentissage), tout en bénéficiant de l'immersion offerte par la réalité augmentée ?

1.3 Objectifs du projet

Le projet NeurAR poursuit les objectifs suivants :

— Concevoir et entraîner un réseau de neurones de type MLP sur le jeu de données MNIST à l'aide de PyTorch ;

— Développer un backend exposant l'état complet du réseau (architecture, activations, poids, biais, métriques) via une API REST et un canal WebSocket temps réel ;

— Concevoir une application Unity en réalité augmentée capable de placer le réseau de neurones dans l'environnement réel et de le représenter en 3D ;

— Animer visuellement la propagation avant (*forward pass*) et la rétropropagation (*backward pass*), couche par couche ;

— Permettre à l'utilisateur d'interagir avec la scène (rotation, zoom, sélection d'un neurone) ;

— Garantir une communication fluide, synchronisée et fiable entre le backend Python et le client Unity.

1.4 Plan du rapport

Ce rapport est organisé comme suit. Le chapitre 2 présente l'architecture générale de la solution. Le chapitre 3 décrit le modèle de réseau de neurones et le backend FastAPI. Le chapitre 4 détaille le protocole de communication temps réel. Le chapitre 5 présente l'application Unity en réalité augmentée. Le chapitre 6 présente les fonctionnalités réalisées et les apports du projet. Le chapitre 7 revient sur les difficultés rencontrées et les solutions apportées. Enfin, le chapitre 8 conclut ce rapport et propose des perspectives d'amélioration.

2. Architecture générale de la solution

2.1 Vue d'ensemble

NeurAR repose sur une architecture **client-serveur** composée de trois grands blocs : le **modèle PyTorch**, le **backend FastAPI** et le **client Unity** en réalité augmentée. La figure 2.1 schématise les échanges entre ces composants.



FIGURE 2.1 – Architecture générale de NeurAR

2.2 Rôle de chaque composant

2.2.1 Le modèle PyTorch

Le modèle est un *Multi-Layer Perceptron* (MLP) entraîné sur le jeu de données MNIST. Il est responsable du calcul des activations, des poids et des biais de chaque couche, ainsi que des métriques d'apprentissage (perte, précision).

2.2.2 Le backend FastAPI

Le backend joue le rôle d'intermédiaire entre le modèle et le client. Il expose :

— une **API REST** pour les requêtes ponctuelles (santé du serveur, architecture du modèle, instantané du réseau, forward pass) ;

— un **canal WebSocket** (`/ws`) pour les interactions temps réel (lancement d'un forward pass ou backward pass, réinitialisation du modèle, diffusion de l'état du réseau couche par couche).

2.2.3 L'application Unity (client AR)

L'application Unity reçoit les données JSON envoyées par le backend, les interprète et construit une représentation 3D du réseau de neurones placée dans l'environnement réel via la réalité augmentée. Elle gère également les interactions de l'utilisateur (rotation, zoom, sélection de neurone).

2.3 Technologies utilisées

Composant	**Technologies**
Modèle de réseau de neurones	Python, PyTorch, torchvision, MNIST
Backend	Python, FastAPI, WebSocket, Pydantic
Client AR	Unity 6000.4.3f1, AR Foundation, C#
Format d'échange	JSON (camelCase, compatible C#)

TABLE 2.1 – Technologies utilisées dans le projet NeurAR

3. Le modèle de réseau de neurones et le backend

3.1 Architecture du réseau (NeuroARNet)

Le réseau de neurones implémenté, nommé `NeuroARNet`, est un **MLP entièrement connecté et configurable**. Son architecture est définie par une simple liste d'entiers (par exemple `[784, 128, 64, 10]`), modifiable via le fichier de configuration `.env`. Pour cette architecture par défaut :

— **Couche d'entrée** : 784 neurones (image MNIST 28×28 aplatie) ;
— **Couche cachée 1** : `Linear(784 → 128)` + `BatchNorm` + `ReLU` ;
— **Couche cachée 2** : `Linear(128 → 64)` + `BatchNorm` + `ReLU` ;
— **Couche de sortie** : `Linear(64 → 10)` + `Softmax` (probabilités des 10 chiffres).

La figure 3.1 illustre cette architecture sous forme simplifiée.

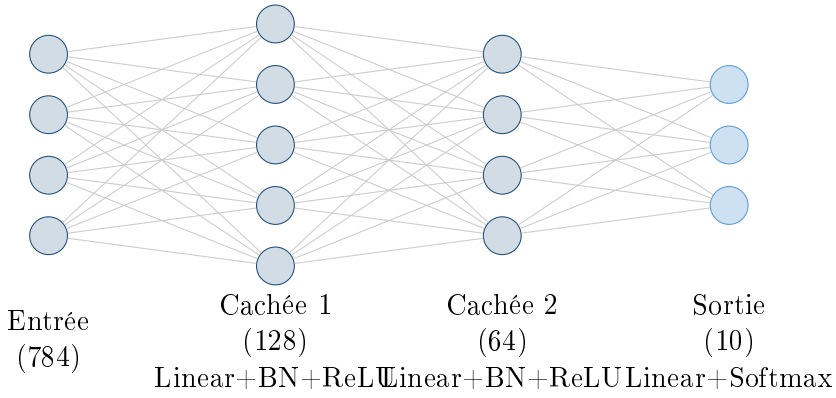


FIGURE 3.1 – Représentation simplifiée de l'architecture du MLP NeuroARNet

3.2 Suivi des activations, poids et biais

À chaque *forward pass*, le modèle conserve les activations de toutes les couches (y compris la couche d'entrée). Le `ModelManager` expose ensuite ces informations sous une forme exploitable par Unity :

— les **activations** de chaque couche sont normalisées entre 0 et 1 (min-max scaling) afin d'être directement utilisées pour l'intensité visuelle des neurones ;

— les **poids** de chaque couche sont normalisés entre -1 et 1 via la fonction tanh, ce qui conserve leur signe tout en bornant leur amplitude pour l'affichage des connexions ;

— les **biais** de chaque neurone sont transmis tels quels.

3.3 Entraînement et métriques

Le modèle est entraîné/optimisé sur des échantillons du jeu de données **MNIST**, téléchargé automatiquement au premier lancement. À chaque *backward pass*, un optimiseur SGD met à jour les poids du réseau à partir d'un échantillon, et les métriques suivantes sont recalculées :

— la **perte** (*loss*), calculée via la *negative log-likelihood* ;

— la **précision** (*accuracy*), égale à 1 si la classe prédite correspond à l'étiquette attendue, 0 sinon.

3.4 Modélisation des données échangées

Pour garantir la compatibilité directe avec les classes C# du côté Unity, les modèles de données Python (Pydantic) génèrent automatiquement des alias en `camelCase`. Trois structures principales sont définies :

— `NeuronData` : un neurone (couche, index, activation, biais, fonction d'activation) ;

— `LayerData` : une couche complète (type, liste de neurones, poids) ;

— `NetworkData` : l'état complet du réseau (nom du modèle, nombre de couches, liste des couches, phase, perte, précision).

3.5 API REST

Le tableau 3.1 résume les points d'entrée REST exposés par le backend.

Méthode	Route	Description
GET	`/health`	Vérifie que le serveur et le modèle sont opérationnels
GET	`/network`	Retourne un instantané complet du réseau
POST	`/forward`	Exécute un forward pass sur une entrée donnée
GET	`/architecture`	Retourne la configuration et le nombre de paramètres

TABLE 3.1 – Points d'entrée de l'API REST

4. Communication temps réel

4.1 Le canal WebSocket

La communication temps réel entre le backend et le client Unity repose sur un canal **WebSocket** (`/ws`). Le client envoie un message JSON contenant un champ `action`, et le serveur répond par un ou plusieurs messages `NetworkData`.

Action	Comportement
`get_network`	Envoie l'état complet du réseau (phase `init`)
`forward_pass`	Exécute un forward pass sur un échantillon MNIST aléatoire et diffuse le résultat **couche par couche** (délai de 100 ms entre chaque couche), puis l'état final complet
`backward_pass`	Exécute un backward pass (mise à jour des poids) et diffuse le nouvel état
`reset`	Réinitialise le modèle (nouveaux poids aléatoires) et diffuse le nouvel état

TABLE 4.1 – Actions supportées par le canal WebSocket

4.2 Scénario d'animation du forward pass

La figure 4.1 présente le scénario d'échange lors d'un *forward pass*. L'envoi couche par couche, avec un léger délai, permet à Unity d'animer progressivement la propagation de l'activation à travers le réseau.

4.3 Gestion de la taille des messages

Le snapshot complet du réseau (poids inclus) représente un volume important : pour une couche $784 \rightarrow 128$, cela correspond à environ 100 000 valeurs, soit environ **2,4 Mo** en JSON. Cette contrainte a un impact direct sur la configuration du client WebSocket Unity (cf. chapitre 7).

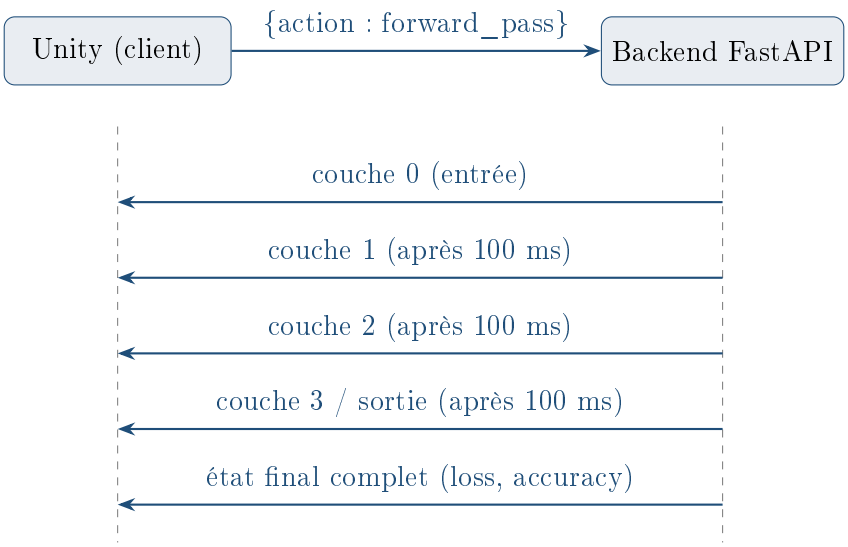


FIGURE 4.1 – Séquence d'échange lors d'un forward pass

5. Application Unity en réalité augmentée

5.1 Vue d'ensemble

L'application client est développée avec **Unity 6000.4.3f1** et le framework **AR Foundation**. Elle est organisée en plusieurs modules spécialisés, listés dans le tableau 5.1.

Module	Rôle
`ARPlacementManager`	Placement du réseau de neurones virtuel dans l'environnement réel
`ARInteractionManager`	Gestion des interactions utilisateur : rotation, zoom, sélection d'un neurone
`WebSocketClient`	Connexion et communication avec le backend via WebSocket
`NetworkDataParser`	Désérialisation des messages JSON reçus en objets `NetworkData`
`UnityMainThreadDispatcher`	Exécution des mises à jour reçues de manière asynchrone sur le thread principal Unity
`NetworkRenderer`	Construction et mise à jour de la scène 3D du réseau
`NetworkLayer`	Représentation d'une couche du réseau
`NeuronVisual`	Représentation visuelle d'un neurone (couleur, intensité selon l'activation)
`ConnectionVisual`	Représentation visuelle d'une connexion (épaisseur/couleur selon le poids)

TABLE 5.1 – Principaux modules de l'application Unity

5.2 Chaîne de traitement côté Unity

1. Le `WebSocketClient` se connecte au backend et envoie les actions demandées (`get_network`, `forward_pass`, etc.) ;
2. Les messages JSON reçus sont transmis au `NetworkDataParser`, qui les convertit en objets `NetworkData` / `LayerData` / `NeuronData` ;
3. L'`UnityMainThreadDispatcher` planifie la mise à jour de la scène sur le thread principal ;

4. Le `NetworkRenderer` met à jour les `NeuronVisual` et `ConnectionVisual` en fonction des nouvelles activations, poids et biais ;
5. L'`ARPlacementManager` et l'`ARInteractionManager` permettent à l'utilisateur de positionner et de manipuler la scène dans son environnement réel.

5.3 Représentation visuelle

Chaque **neurone** est représenté par un objet 3D dont la couleur ou l'intensité lumineuse reflète sa valeur d'activation normalisée. Chaque **connexion** entre deux neurones est représentée par un segment dont l'apparence (couleur, épaisseur) reflète le poids associé (normalisé entre -1 et 1). Lors d'un forward pass, les couches s'illuminent successivement, donnant une animation visuelle de la propagation de l'information à travers le réseau.

6. Fonctionnalités réalisées et apports du projet

6.1 Fonctionnalités réalisées

— Visualisation du réseau de neurones, couche par couche, en 3D et en réalité augmentée ;

— Animation du **forward pass** : propagation visuelle de l'activation à travers les couches ;

— Animation du **backward pass** : mise à jour visuelle des poids après apprentissage ;

— Affichage en direct des valeurs d'activation, des poids et des biais de chaque neurone ;

— Suivi en temps réel de la **perte (loss)** et de la **précision (accuracy)** du modèle ;

— Réinitialisation du modèle à la demande (nouveaux poids aléatoires) ;

— Interactions en réalité augmentée : placement de la scène, rotation, zoom et sélection d'un neurone pour afficher ses détails.

6.2 Apports et points forts du projet

— **Approche pédagogique innovante** : le projet rend visible et manipulable le fonctionnement interne d'un réseau de neurones, habituellement abstrait ;

— **Architecture découplée et réutilisable** : le backend FastAPI est indépendant du client et pourrait servir d'autres applications (web, mobile) sans modification ;

— **Communication temps réel performante** grâce au WebSocket et à la diffusion couche par couche ;

— **Modèle configurable** : l'architecture du MLP est modifiable simplement via le fichier `.env`, sans modification du code ;

— **Compatibilité stricte avec Unity/C#** grâce à la génération automatique d'alias `camelCase` dans les modèles Pydantic ;

— **Expérience immersive** en réalité augmentée, utilisable sur smartphone ou casque AR compatible ;

— **Projet transversal** couvrant trois domaines complémentaires : intelligence artificielle (PyTorch), développement backend (FastAPI) et applications immersives (Unity / AR Foundation).

7. Difficultés rencontrées et solutions apportées

— **Volume de données important** : un instantané complet du réseau (poids inclus) représente environ 2,4 Mo en JSON.
Solution : augmentation de la limite de taille des messages acceptés par le client WebSocket Unity (`NativeWebSocket` / `websocket-sharp`) pour éviter toute déconnexion lors de la réception du réseau complet.

— **Synchronisation entre le calcul backend et l'animation Unity** : un envoi instantané de toutes les couches rendrait l'animation illisible.
Solution : diffusion des couches une à une, avec un délai contrôlé de 100 ms entre chaque couche, pour produire une animation fluide et compréhensible.

— **Alignement strict entre les modèles Python et les classes C# Unity** : toute différence de nommage entre les champs JSON et les propriétés C# empêche la désérialisation correcte.
Solution : mise en place d'une classe de base `CamelModel` générant automatiquement des alias `camelCase` pour chaque champ, garantissant une correspondance exacte avec `NetworkData`, `LayerData` et `NeuronData` côté Unity.

— **Stabilité du modèle en mode évaluation lors du backward pass** : l'utilisation de `BatchNorm` sur un batch de taille 1 peut poser problème.
Solution : conservation du mode `eval()` du réseau, ce qui permet à `BatchNorm` d'utiliser ses statistiques courantes (*running stats*) tout en laissant les gradients se propager normalement pour la mise à jour des poids.

— **Mise en place de l'environnement de réalité augmentée** : configuration de AR Foundation, gestion multi-plateforme (Android/iOS) et tests sur appareil physique, nécessaires pour valider le placement et le suivi de la scène en conditions réelles.

8. Conclusion et perspectives

8.1 Conclusion

Le projet **NeurAR** a permis de concevoir une chaîne complète allant de l'entraînement d'un modèle de réseau de neurones avec PyTorch, jusqu'à sa visualisation interactive en réalité augmentée via Unity, en passant par un backend temps réel basé sur FastAPI et WebSocket. Il répond à une problématique pédagogique concrète : la difficulté de comprendre le fonctionnement interne des réseaux de neurones, en proposant une expérience immersive, visuelle et interactive.

Les principaux résultats obtenus sont :

— un modèle MLP fonctionnel, configurable et entraîné sur MNIST ;

— un backend robuste exposant l'état du réseau via REST et WebSocket, avec une animation couche par couche ;

— une application Unity AR capable d'afficher et d'animer le réseau en 3D, avec des interactions utilisateur (rotation, zoom, sélection).

8.2 Perspectives d'amélioration

Plusieurs pistes permettraient d'enrichir le projet :

— **Support d'autres architectures** de réseaux de neurones (réseaux convolutifs CNN, réseaux récurrents RNN) ;

— **Mode multi-utilisateurs** permettant à plusieurs personnes de visualiser et manipuler la même scène AR simultanément ;

— **Entraînement interactif** : permettre de modifier les hyperparamètres (taux d'apprentissage, architecture) directement depuis l'application et observer l'impact en direct ;

— **Optimisation de la taille des données transmises**, par exemple via la compression ou l'envoi de mises à jour différentielles (*delta-update*) plutôt que l'état complet à chaque étape ;

— **Déploiement sur casques AR/VR autonomes** (HoloLens, Meta Quest) pour une expérience totalement immersive et mains libres.